



Transcript

Week 29: Reinforcement Learning

Video 1 – Reinforcement Learning

RL is inspired by psychology. Just like humans learn from experiences, machines can too. That's why reinforcement learning is so powerful in situations where the right decision isn't clear from the start, like training a robot to walk, teaching an AI to play games, or even helping self-driving cars figure out how to handle traffic

Reinforcement Learning, or RL, is a method that helps machines learn through experience, not by following step-by-step rules.

Imagine a child trying to ride a bicycle. At first, they wobble or even fall—that is the penalty. But when they manage to balance and ride smoothly, that is the reward. Over time, by repeating this process, the child figures out the best way to ride.

This is how RL works. You have an agent that interacts with its environment. Each action it takes allows it to receive feedback—either a reward for doing well or a penalty for not. The agent then uses this to adjust and improve decisions over time.

Instead of being told the right answer, the agent discovers answers through trial and error, learning strategies that maximise rewards in the long run.

And here is the fascinating part—RL is designed to draw inspiration from human psychology. Just as people learn from experience, so can machines. That is why RL is so powerful, especially when applied to robots, games, and self-driving cars.

Now let's talk about the two main types of reinforcement learning: Model-Free and Model-Based.

In Model-Free RL, the agent simply jumps straight in and learns directly from its experience. Rather than building a model of the world, it tries different moves, sees what happens, and keeps learning through trial and error. This is much like learning a new video game by experimenting, without reading the manual. The system keeps improving as it goes along. Techniques like Q-learning and Deep Q-Networks fall under Model-Free RL and are often used for game-playing AIs.

On the other hand, Model-Based RL means the agent takes the time to build a mental map or a model of how things work. With this, it's able to simulate future scenarios, plan its strategy, and make more informed decisions. Imagine playing chess and considering how your opponent might respond before you make your move. This approach includes methods like Monte Carlo and planning-based reinforcement learning.

So, to connect these ideas, Model-Free RL learns by doing—quick to start but less strategic. Model-Based RL learns by imagining and planning—more effortful, but thoughtful and strategic. Each has its place depending on whether you need speed or careful planning in reinforcement learning.



Reinforcement Learning, or RL, is different from other learning types such as Supervised and Unsupervised, because each one solves a different kind of problem.

In Supervised, the system learns from labelled data. The goal is to apply this to classification and regression tasks, predicting the right output for new inputs.

In Unsupervised, the data has no labels. The model tries to find patterns, and we apply it to clustering and reduction problems.

In Reinforcement Learning, the agent learns from actions and feedback. The aim is to maximise rewards over time, which makes it especially useful when applied to robots, games, and autonomy.

So while supervised and unsupervised methods focus on data, RL focuses on interaction and experience.

Reinforcement Learning, or RL, is what makes artificial intelligence truly powerful. Let's explore the three key reasons why:

First, Autonomous Decision-Making. RL allows systems to learn from their environment and act independently. Instead of relying on hard-coded rules, they figure out the best actions through experience. For example, a self-driving car learns when to slow down, change lanes, and avoid obstacles, all on its own.

Second, Exploration and Exploitation. This core strength of RL balances trying new actions, called *exploration*, with sticking to strategies that already work, known as *exploitation*. Think of trying a new dish at a restaurant versus ordering your favorite—you need both to improve over time.

Third, Adaptability. RL systems can adjust to changing environments. Unlike traditional AI that may fail with new conditions, RL agents continue learning. A trading bot or warehouse robot, for example, adapts to sudden changes and unexpected events, getting smarter with every interaction.

In summary, RL empowers AI to act independently, improve continuously, and stay flexible in dynamic situations, making it a truly powerful approach.

Reinforcement Learning is transforming robotics.

Robots are becoming smarter and more adaptive thanks to RL.

In Robot Navigation, robots learn to move safely through unknown environments, finding paths and avoiding obstacles on their own.

With Autonomous Manipulation, robots learn to handle and manipulate objects, picking up, stacking, or assisting in delicate tasks effectively.

Through Adaptive Control, robots adjust their movements to stay stable and efficient in changing conditions, like walking on uneven ground or flying.

Overall, RL is making robots smarter, more capable, and able to continuously learn from their environment. Reinforcement Learning has truly shined in game playing



providing a perfect testing ground for decision-making strategy and trial-and-error learning.

AlphaGo developed by DeepMind stunned the world by defeating Go champions despite the game's complexity learning creative strategies beyond human imagination.

Atari Games trained using Deep Q-Learning started with zero knowledge and learned to play classics like Breakout and Space Invaders at superhuman levels discovering clever techniques along the way.

Chess and Poker AI systems like AlphaZero and Libratus mastered chess and poker teaching themselves strategies that outsmarted top engines and professional players.

Reinforcement Learning is not just about playing games it pushes the limits of what machines can learn often in ways that surprise even the experts.

Reinforcement Learning is at the heart of many autonomous systems emerging around us today operating with minimal human intervention in complex and ever-changing environments.

Self-Driving Cars use RL to navigate city traffic highways and unexpected obstacles learning when to speed up brake and change lanes safely.

Drone Navigation allows drones in delivery services and disaster management to find the best routes avoid collisions and cover large areas efficiently adapting to rough terrains.

Smart Cities apply RL to optimise urban systems including traffic management energy consumption and resource allocation creating intelligent adaptive cities.

Whether it is cars on the road drones in the sky or cities of the future Reinforcement Learning is driving autonomy to the next level.

Reinforcement Learning is not just limited to laboratories or games it is already making a difference in real-world systems that impact our daily lives.

In Healthcare RL is used to design personalised treatment plans helping doctors choose the best therapies for individual patients it is also applied in drug discovery allowing RL agents to test countless chemical combinations faster than humans and in robotic surgeries RL helps surgical robots perform precise delicate operations improving continuously through experience.

In Finance RL is transforming how money is managed it supports portfolio optimisation helping investors balance risks and returns it powers algorithmic trading systems that make split-second decisions and is also used in fraud detection learning to spot unusual patterns that could indicate fraud.

In Energy Systems RL plays a big role in optimising power grids and making them smarter it reduces energy waste by automatically adjusting supply to match demand and manages renewable sources like solar and wind more efficiently enabling more sustainable and reliable energy.



Whether it is improving healthcare outcomes making financial systems smarter or creating sustainable energy solutions reinforcement learning is driving real-world impact across industries.

While reinforcement learning is powerful it is not without challenges let us look at some of the main hurdles researchers and practitioners face.

Exploration versus Exploitation RL agents have to balance trying new actions and sticking with strategies that already work it is like ordering your favourite dish versus trying something new you might find something better or be disappointed.

Sample Efficiency RL agents need large amounts of interaction data to learn effectively training a robot might require millions of trial-and-error attempts making learning time-consuming and computationally expensive.

Reward Shaping Agents depend on rewards to guide behaviour designing proper rewards is tricky without them agents might learn unintended or harmful behaviours it is like teaching a child you have to be careful what behaviours you encourage.

Long-Term Delayed Rewards Agents struggle to connect actions with outcomes when rewards come much later it is like exercising today and only seeing results months later environments with rare or delayed rewards make learning harder.

Despite these challenges reinforcement learning has huge potential and remains an active area of research and development.

As powerful as reinforcement learning already is the future holds even more exciting possibilities let us look at some key directions where RL is headed.

Generalised RL Currently RL agents are trained for very specific tasks The challenge is to develop agents that can transfer skills from one task to another like a robot learning tennis and then applying that knowledge to badminton.

Multi-Agent RL Involves multiple agents interacting in the same environment For example self-driving cars need to cooperate with others to avoid accidents while also competing for space Multi-agent RL enhances collaboration and competition among agents making them smarter in shared environments.

Human-Robot Interaction Future RL agents will need to work alongside humans Imagine a healthcare robot assisting in surgeries while communicating effectively with doctors or household robots learning your habits to help more intuitively This direction improves collaboration safety and effectiveness.

The future of RL is about making it more adaptable collaborative and human-aware pushing us closer to intelligent systems that integrate into everyday life.

Imagine what you could create if you applied reinforcement learning to solve real-world problems what will you build next?



Video 2 - Reinforcement Learning: Core Concepts

How does a robot learn to reach its goal or a game player get better with every move? Reinforcement learning reveals how agents learn through interaction and reward. Let's break down the key ideas that make intelligent decisions possible.

To really understand reinforcement learning it helps to break it down into its key components let us go through them one by one

First we have the Agent This is the learner or the decision-maker Think of it as the player in a game it is the one that takes actions and learns from the outcomes

Next is the Environment This is everything the agent interacts with If the agent is the player then the environment is the game world itself it provides situations challenges and feedback

Then comes the State A state is simply a snapshot of the environment at a particular moment in time For example in chess a state would be the exact arrangement of all the pieces on the board at a given turn

The Action Refers to the choices made by the agent These are the moves or decisions that affect what happens next In our chess example moving a knight or a pawn would be an action

The Reward Is the feedback the agent gets after an action It is usually a number that tells the agent how good or bad its move was For instance scoring a goal in football could be a positive reward while losing possession might be a negative one

Next is the Policy This is like the agent's playbook a strategy that tells it what action to take in each state Over time the policy improves as the agent learns which actions lead to better rewards

And finally the Value Function While the reward tells the agent about the immediate benefit of an action the value function measures the long-term benefit It helps the agent think ahead considering not just the next move but the future rewards it might lead to.

Together these components agent environment state action reward policy and value function form the backbone of reinforcement learning allowing machines to learn and adapt through experience.

An agent is the learner or decision-maker in an R-L system. It is responsible for interacting with the environment by taking actions and receiving feedback based on those actions. The agent's goal is to maximise the cumulative reward over time by improving its decision-making.

The role of the agent is to explore the environment by performing actions, observe the outcomes of those actions, and then adjust its behaviour based on the feedback received. Feedback usually comes in the form of rewards, which is positive feedback, or punishments, which is negative feedback. The agent learns from this feedback and refines its policy to make better decisions in the future.



There are different types of agents. In many R-L scenarios, a single agent interacts with the environment. For example, in robotics, the robot itself is the agent that takes actions like moving or grasping. In multi-agent systems, multiple agents interact with each other and the environment. This is common in situations like game playing.

A classic example of an agent in R-L can be found in the game of chess. In this case, the player is the agent. The player makes moves based on the current state of the board, and the feedback—rewards or penalties—depends on the outcome of each move.

In reinforcement learning, the environment plays a crucial role as it is the context or the world within which the agent operates and interacts. The environment provides feedback to the agent, influencing how the agent learns and adapts its decision-making.

To begin with, the environment refers to everything the agent interacts with and tries to influence. It includes any external system in which the agent operates, such as a game world, a factory, or a city grid.

Building on this, the role of the environment is to respond to the agent's actions. When the agent makes a move, the environment changes its state and gives feedback, usually as rewards or punishments. This ongoing feedback process helps the agent evaluate how effective its actions are and to improve its behaviour over time.

Let's look at the interaction between the agent and the environment. Each action taken by the agent can alter the environment's state—such as its position, temperature, or road and traffic conditions. In response, the environment updates this state and provides a reward depending on how favourable the action was. This feedback loop continues, guiding the agent to adjust its strategy with every move.

For example, think about a self-driving car. In this scenario, road and traffic conditions are part of the environment. The car makes decisions like turning, accelerating, or braking, and the environment supplies feedback—such as rewards for arriving safely or penalties for mistakes. Over time, this process teaches the agent to adapt, so it can navigate more effectively and safely.

In summary, the environment is a dynamic system the agent interacts with and influences. The feedback received from the environment is what drives learning in reinforcement learning. The better the agent understands and adapts to the environment, the more successful it will be in reaching its goals.

In reinforcement learning, the state is a fundamental concept that represents the current situation or configuration of the environment at a given point in time. The state provides crucial information to the agent, helping it decide what action to take next.

A state is a representation of the environment's current situation. It captures all the relevant aspects of the environment that the agent needs to consider when deciding which action to take. The state reflects the context within which the agent operates and makes decisions.



The role of the state is to provide the agent with the necessary information to make decisions. It serves as the input that the agent uses to choose an appropriate action. By understanding the current state of the environment, the agent can evaluate its options and select actions that are likely to lead to favourable outcomes.

There are two main types of states in reinforcement learning. First, discrete states consist of a finite set of possibilities. For example, a board in chess represents a finite arrangement of pieces, and each arrangement is a distinct state. Second, continuous states involve an infinite or uncountable number of possibilities. This could be the position or speed of a robot moving in physical space, or the situation of a robot in a factory—these states are defined by variables that can change in many ways.

To illustrate, in a video game setting, the state could include the player's current position, health status, and resources. All these details show where the player is within the game world and what choices they can make. The agent uses this state information to decide what actions to take, such as moving, attacking, or using an item.

In summary, the state is a crucial piece of information that defines the current situation of the environment from the agent's perspective. By understanding the state, the agent can decide which action will best help it achieve its objectives, whether that's winning a game, reaching a goal, or optimising some performance measure.

In reinforcement learning, the action is a critical component that drives the interaction between the agent and the environment. These actions are the choices made by the agent to influence the state of the environment, allowing it to work towards its goals.

To start, an action is a choice the agent makes in order to interact with or change the environment. The agent decides on actions based on its current state, and these actions help transition it from one situation to another. The agent's objective is to take actions that maximise rewards and help reach its goals.

Building on this, the role of actions is to enable the agent to move through the environment and change its state. When the agent acts, the environment responds and provides new information—this helps the agent make further decisions and improves its strategy over time. By choosing the right action at the right moment, the agent can maximise the rewards it receives throughout its journey.

Now, let's look at the types of actions in reinforcement learning. Some environments use discrete actions, which means the agent chooses from a finite set of options, like moving up, down, left, or right in a grid-based game. Other settings involve continuous actions, where the agent has infinite possibilities—such as adjusting the steering angle or speed of a car. These continuous actions, common in robotics and autonomous driving, require smooth adjustments and precise control.

For example, imagine a self-driving car. It can choose to turn left, turn right, or accelerate. Every choice influences the car's movement and helps it navigate safely through the environment. The car's control system makes decisions based on its



current state—each action helps it reach its destination, avoid obstacles, and stay safe.

In summary, action represents the decision-making process at the core of the agent's behaviour. It's how the agent interacts with the environment and transitions between different states. Whether actions are discrete or continuous, the agent's ability to act effectively is what enables it to achieve its goals and maximise long-term rewards in reinforcement learning.

In reinforcement learning, the reward is a vital component that helps guide the agent's behaviour and learning process. Rewards serve as the agent's feedback mechanism, informing the agent whether its actions lead it toward its goal or away from it.

A reward is the feedback the agent receives after performing an action. It indicates the success or failure of the agent's actions and provides a value that reflects how well those actions align with the agent's objective. In reinforcement learning, rewards are used to reinforce good behaviour and discourage actions that lead to poor outcomes. Positive rewards signal progress in the right direction, whereas negative rewards show the opposite.

The role of rewards is to shape the agent's learning process. The agent seeks to maximise its cumulative reward over time, and rewards help it decide which actions are useful and which are best avoided. Positive rewards encourage the agent to repeat actions that result in desirable outcomes. Negative rewards, or punishments, discourage the agent from repeating actions that lead to unwanted results.

Looking at the properties of rewards, we find three main types. Positive rewards encourage the agent to repeat beneficial actions. Negative rewards, or punishments, discourage the agent from actions that result in negative outcomes. Delayed rewards are common in reinforcement learning and are not always immediate; sometimes, agents must wait for more significant rewards that can accumulate through a sequence of actions.

For example, in a game, scoring points after completing a task is an example of a positive reward and motivates the agent to repeat that action in the future. On the other hand, losing health is an example of a negative reward, which discourages the agent from repeating the same mistake.

In summary, the reward is an essential mechanism in reinforcement learning, helping the agent learn which behaviours to repeat and which to avoid. By planning for both immediate and delayed rewards, the agent can make better decisions and achieve its goal more effectively.

In reinforcement learning, the policy plays a crucial role in guiding the agent's decisions by providing a strategy for what actions to take in different states of the environment.

A policy is the strategy or rule the agent uses to decide its actions based on the current state. It is essentially the agent's decision-making process—mapping states to actions and helping the agent interact with the environment in a meaningful way to maximise rewards over time.



Moving on, the role of the policy is to direct the agent's behaviour and influence the results it achieves. The policy is central to the agent's learning process. By following and refining its policy, the agent can make better choices to pursue long-term goals and success in various situations.

There are two main types of policies in reinforcement learning. First, a deterministic policy always gives the same action for a given state. It's predictable and fixed—for example, in chess, a deterministic policy might always move the same piece under identical board conditions. Second, a stochastic policy introduces randomness, sometimes choosing different actions in the same situation, according to a probability distribution. This allows for more exploration. In a game like Pac-Man, for instance, the policy might sometimes choose a less direct route to try something new.

For example, in chess, the policy could be a set of rules or a learned strategy that the agent follows to decide moves based on the board's configuration. The policy can evolve as the agent learns and adapts from experience.

In conclusion, the policy is the fundamental decision-making strategy in reinforcement learning. Whether deterministic or stochastic, it shapes the agent's behaviour, helps improve performance, and supports the agent in achieving its long-term goals.

In reinforcement learning, the value function plays a crucial role in guiding the agent's decisions. It helps the agent evaluate which states or actions are most beneficial in the long run.

The value function measures the expected reward an agent can achieve from a particular state, s , or action, a , considering both immediate and future rewards. The goal is to estimate how good a given state or action is, based on the rewards it leads to over time.

The state value, written as V of s , is the expected reward starting from state, s . It helps the agent decide how advantageous it is to be in that state, guiding planning for maximum reward.

Building on this, the action value, called Q of s comma a , measures the expected reward from taking a specific action, a , in a particular state, s . This lets the agent evaluate not just being in a state, but the value of different actions it can take from that state, helping it to choose the best action at every step.

Policy improvement refers to the process by which the agent refines its strategy or policy to maximise the expected reward. The agent continually adjusts its policy to choose actions that maximise the value function, improving its decision-making over time. This ongoing improvement is central to algorithms like Q-learning and Policy Gradient methods.

Finally, the agent's main objective is to maximise cumulative rewards, balancing immediate and future gains. For this, the agent uses a discount factor to decide how much it values future rewards compared to immediate rewards—a higher discount factor gives more importance to the future, while a lower one focuses on short-term gains.



By refining the value function and improving the policy, the agent gradually learns the optimal behaviour to achieve the highest total reward over time.

In reinforcement learning, the core ideas—agent, environment, state, action, reward, and policy—interact in a loop that enables the agent to learn and perform better over time.

Everything begins when the agent observes the state of the environment. This state gives the agent a snapshot of what's currently happening, like a car's location and speed or a player's position in a game.

By consulting its policy—a strategy or set of rules for making decisions—the agent then selects an action. The policy essentially maps each state to specific actions, helping the agent decide what to do in any situation it encounters. Policies may be deterministic, always choosing the same action for a given state, or stochastic, introducing some randomness by selecting actions based on probabilities. Through learning and exploration, the agent aims to refine its policy to guide it toward the best possible decisions.

After an action is taken, the environment reacts by updating to a new state and offering feedback in the form of a reward or penalty. This feedback tells the agent how successful its move was, such as giving points for positive actions or applying penalties for mistakes.

With the new state and reward in mind, the agent uses policy improvement to update its strategy. By evaluating which actions and strategies led to good results, the agent continuously adjusts its policy. The goal is to improve decisions and maximise cumulative rewards over time.

This cycle of observing, deciding, acting, and learning repeats again and again. By refining its policy and learning from rewards, the agent gradually develops more effective behaviours for achieving its objectives in any environment.

Let's walk through a simple example of reinforcement learning: a robot that learns how to reach its goal in a room.

We start by identifying the agent. Here, the agent is the robot itself—the learner that makes decisions. The learning process begins when the robot starts from a random position in the room.

Next, the environment is defined. The environment is simply the room where the robot is placed. This is the physical space the robot must navigate in order to reach its goal.

Then, we determine the state. At any moment, the robot's position is called the state. This state is a snapshot, allowing the robot to understand where it is and what's around it.

The robot uses its policy—a set of rules for decision-making—to choose an action. It can move forward, left, or right, depending on where it is and what the policy recommends. For example, if the goal is positioned to the right, the robot may decide to move right.



After executing an action, the robot receives feedback. A move towards the goal earns a positive reward, but bumping into a wall or moving away gets a negative reward. This feedback helps the robot learn which moves are better.

With each move, the robot updates its policy. This means it refines its rules for which actions to repeat and which to avoid in the future. Through this continual adjustment, the robot's strategy becomes smarter and more effective.

This cycle—starting, observing, choosing, moving, receiving feedback, and improving the policy—repeats again and again. Over time, by trial and error, the robot steadily becomes better at navigating to its target, drawing on the rewards it receives to guide smarter decision-making.

Let's finish by summarising the core concepts in reinforcement learning.

The agent is the decision-maker in the RL system. It learns how to act by making decisions as it interacts with the environment.

The environment is the external system or context where the agent operates and interacts. It responds to the agent's actions, providing feedback for learning.

State refers to a specific representation of the environment's condition at a particular moment in time. The agent uses this information to decide what to do next.

Action means the choices the agent makes to influence or change the state of the environment. These actions help the agent achieve its goals.

Reward is the feedback the agent receives after it performs an action. The reward helps the agent understand how good or bad the action was in achieving its goal, guiding its future learning process.

Finally, the policy is the strategy or rule the agent follows to decide which action to take, given its current state. The policy essentially shapes the agent's behaviour as it learns how to perform better within different situations.

Together, these concepts—agent, environment, state, action, reward, and policy—form the foundation of reinforcement learning, helping the agent to continually learn and improve its decision-making skills.

Ready to put learning into action? Try building your own smart agent, experiment with rewards, or team up with others and see who can create the cleverest learner. Start exploring reinforcement learning today!

Video 3 – Reinforcement Learning Algorithms

Ever wondered how a self-driving car learns to navigate traffic or how a robot learns to perform a complex task? They learn from experience, using a powerful family of algorithms called Reinforcement Learning. In this video, we'll explore the core methods that teach AI to learn by doing.

When it comes to reinforcement learning, we can broadly divide the methods into two main categories: Model-Free and Model-Based.



Let's start with Model-Free Methods. The core idea here is to learn directly from trial and error without building a mental map of the environment. You just figure out what actions get you the best results through repeated interaction. Examples of this approach include Q-learning and SARSA.

To put it simply, Model-Free is about 'learning by doing', while Model-Based is about 'learning by imagining'. Both have their strengths, with Model-Free often being simpler to start, while Model-Based can be more efficient and strategic.

Value-based methods in reinforcement learning aim to estimate the value function to determine the optimal policy. The core idea is that the agent uses value functions to guide decision-making, rather than directly learning the policy. These methods focus on learning the optimal value function and then deriving the policy based on it. The value function helps in evaluating how good a particular state or action is by estimating the expected cumulative reward the agent can achieve from that state or action.

Value-based methods generally focus on estimating the value of each state or action. Once the value function is learned, the agent can then select actions that maximise the expected value. These methods are generally well-suited for problems where the state space is large but can be broken down into discrete state-action pairs, allowing for efficient learning of the optimal values.

Examples of Value-Based Methods include Q-learning and Deep Q-Networks, or DQN. Q-learning is a model-free, off-policy algorithm that learns the value of action-state pairs, or Q-values. The Q-values are updated based on the reward received and the estimated future values, aiming to maximize the expected cumulative reward. Deep Q-Networks is an extension of Q-learning that uses deep neural networks to approximate Q-values. DQN introduces techniques like experience replay and target networks to improve stability and efficiency, and enables the algorithm to handle high-dimensional state spaces.

Policy-based methods directly learn the optimal policy by parameterizing a policy function and optimising it, without needing a separate value function. The policy function defines the mapping from states to actions, which the agent then follows to maximise cumulative rewards over time.

Unlike value-based methods that estimate the value of states or actions, policy-based methods focus on learning the policy directly. The agent adjusts its policy to select actions that will maximise the expected reward over time. The primary focus of these methods is to optimise the policy directly, often using gradient-based approaches.

The policy also directly specifies the probabilities of taking different actions in a given state, and the goal is to find the policy that maximises long-term rewards.

Examples of policy-based methods include REINFORCE, a Monte Carlo-based algorithm that uses the policy gradient method to directly update the policy. Another is Proximal Policy Optimisation, or PPO, a more advanced and stable policy gradient method that optimises the policy using a surrogate objective function. PPO helps avoid large, destabilising policy updates by restricting the change in policy at each step. Finally, Trust Region Policy Optimisation, or TRPO, is a method designed



to ensure that each policy update remains within a 'trust region' to maintain stability.

These methods are particularly effective when the action space is continuous or the problem requires fine-grained control, such as in robotic control or complex decision-making environments.

Now let's talk about Actor-Critic methods, which are a very popular family of reinforcement learning algorithms. The first thing to understand is that the Actor-Critic method is a hybrid approach; it combines the strengths of both value-based methods and policy-based methods, which directly learn strategies. By bringing these two together, we get a much more stable and efficient learning process.

Here's how it works: the Actor is the part of the system that learns the policy—it decides which action to take in a given situation. Think of the actor as the decision-maker, or the one making moves in a game. Then we have the Critic. The critic evaluates how good those actions are by estimating a value function. In other words, the critic gives feedback to the actor, saying: 'That was a good move' or 'That wasn't the best choice.' This constant interaction between actor and critic allows the system to improve quickly.

Some well-known examples of this approach are Advantage Actor-Critic, or A2C, and Deep Deterministic Policy Gradient, or DDPG, which are widely used in advanced reinforcement learning applications, like robotics and continuous control problems. So, in short, the Actor-Critic method is like having a player who makes moves, paired with a coach who evaluates and guides them, creating a powerful learning loop.

Model-Free algorithms are especially useful in situations where building a complete model of the environment is either impossible or too complicated.

First, they shine in Unknown Environments, where the environment is not known and cannot be directly simulated. For example, a Mars rover exploring the surface of the planet doesn't have a perfect model of the terrain, so it must use trial and error to decide how to move safely and avoid hazards.

They are also highly effective in Real-world Scenarios. Many real-world environments are extremely complex, with rules that are not fully known or constantly changing. A good example here is financial trading. Stock markets are influenced by countless unpredictable factors, making it nearly impossible to model everything. Model-Free algorithms can learn profitable strategies directly by interacting with the market, without a perfect understanding of its dynamics.

And finally, they are excellent for Adaptability. Model-Free methods are great at adapting to changing conditions without needing a pre-defined model. For example, self-driving cars use reinforcement learning to adapt to traffic patterns, weather conditions, or unexpected obstacles. They do not rely on a fixed set of rules but instead learn and adapt through continuous interaction with their environment.

So, whether it's navigating unknown worlds, dealing with complex real-world systems, or adapting on the fly, Model-Free algorithms provide practical solutions where building a model is just not feasible.



Model-Based algorithms are especially powerful when we have environments that can be accurately modelled or simulated. They are valuable when planning, predictability, and efficiency are the most important factors.

First, these methods are excellent for Planning and Simulation. Model-Based reinforcement learning is very effective when we can simulate different actions before trying them in the real world, as this leads to much faster learning. A great example of this is using flight simulators to train pilots.

Next, they are ideal for Predictable Environments. These algorithms shine in scenarios where the rules and dynamics are stable and can be accurately modelled and predicted, such as in supply chain optimisation where companies can model their transportation routes and warehouse logistics to minimise costs and delays.

Finally, Model-Based algorithms are great for Sample Efficiency. Since these methods can learn by simulating outcomes, they often need fewer real-world interactions, which makes them more efficient. A perfect example is in robotics in healthcare, like robotic surgery, where it would be impractical for a robot to learn purely by trial and error on real patients.

Policy-Based and Actor-Critic methods are especially powerful when we deal with continuous action spaces—situations where actions are not just simple yes-or-no choices, but involve fine-grained, ongoing control.

The first key application is Robotics Control. These methods are widely used in robotic manipulation and locomotion tasks. For example, in teaching a robotic arm to carefully pick up delicate objects, since these movements are continuous and precise, policy-based and actor-critic approaches are a perfect fit.

Next, we have Autonomous Vehicles. Self-driving cars operate in complex, dynamic environments where decisions must be continuous and real-time. Actor-Critic methods such as DDPG, or Deep Deterministic Policy Gradient, are here because they balance decision-making with ongoing evaluation, allowing vehicles to adapt to constantly changing conditions.

And finally, Game AI. These methods have been game-changers for creating intelligent agents in complex strategy games and simulations. For example, in real-time strategy games like StarCraft, the AI has to manage resources, control multiple units, and plan ahead. Actor-Critic methods allow such agents to make nuanced, multi-step decisions that go far beyond simple trial-and-error strategies.

So overall, Policy-based and Actor-Critic methods excel in environments where actions are continuous, precise, and highly dependent on context—making them ideal for robotics, autonomous vehicles, and advanced game AI.

Now that you've seen how these algorithms learn to act, think, and strategise, it's your turn. How will you shape the future of AI?

Video 4 – Q Learning

Imagine you're dropped in a maze with no map, no instructions, and every wrong turn costs you. How would you figure out the best path to the exit—quickly and efficiently? That's exactly what Q-Learning teaches robots and algorithms to do. In



this video, you'll discover how simple trial and error can help machines (and even you!) master everything from games to real-life navigation—one step at a time.

Let's simplify one of the most famous reinforcement learning algorithms: Q-Learning. Think of it like this: an agent is in a maze and it has no map. It has to figure out the best way to get to the exit all on its own.

Q-Learning is a model-free algorithm, which means it learns by trial and error without needing a pre-built model of the world. Its goal is to find the most optimal path to a goal by continuously trying different actions.

The core concept is the Q-value. Every time the agent takes an action, it gets feedback—a reward or a penalty. The agent uses this feedback to update a Q-value for that specific action, essentially learning how good it was. Over time, it builds up a Q-table of these values, which serves as a memory of what works and what doesn't. It will always choose the action with the highest Q-value, eventually figuring out the best path without ever knowing the map in advance.

Now that we know what Q-Learning is, let's look at how the algorithm actually works. At the heart of Q-Learning is the Q-function. This function represents the value of taking a certain action, let's call it 'a', in a given state, 's'. It helps the agent decide which action will bring the highest long-term reward.

The learning itself happens through what's called the Update Rule. This is the core formula the agent uses to update its knowledge. In simple terms, the update rule says: take the old Q-value, and adjust it using the reward you just got, plus the best possible future reward you can expect. As this process repeats, the Q-values converge towards the optimal strategy.

Now, let's break down the ingredients of the Update Rule. First, we have alpha or the learning rate. This controls how much weight we give to new information compared to what we already know. Next is 'r', which stands for the reward received. This is the immediate feedback the agent gets after taking an action. Then we have gamma or the discount factor. This tells us how much importance we give to future rewards compared to immediate ones. And finally, we have 's prime', which is the next state. After the action, the environment changes, and the agent ends up in this new state. Every step it takes updates its knowledge: if it moves closer to the exit in our maze example, its Q-values increase for that action. If it hits a wall, the reward is negative, and the Q-value for that action decreases. Eventually, the robot learns the best path to reach the goal efficiently.

One of the main goals in Q-Learning is to find the right balance between trying new things and sticking with what already works. The agent has to be curious and explore different actions to discover better options, but it also needs to make smart choices based on what it already knows gives good results.

To keep that balance, Q-Learning uses what's called an epsilon-greedy policy. Most of the time, the agent chooses the best-known action, but every so often, it makes a random move. This simple approach helps the agent avoid getting stuck making the same choices, so it can keep learning and improving.

Q-Learning isn't just used in theory; it's found in a lot of real-world areas. In games, it helps computers get better at playing things like Tic-Tac-Toe, and it's



even used in more complicated game strategies. In robotics, Q-Learning teaches robots to find their way through mazes, keep their balance, or control objects—without needing a perfect picture of their environment. In finance, Q-Learning helps computers make smarter investment decisions by teaching them to weigh the benefits of safe bets and new opportunities.

Overall, Q-Learning is about making smarter decisions by mixing curiosity and experience. When we apply these ideas in practice, we can tackle real problems and improve by learning from both our mistakes and our successes.

So how does Q-Learning actually work? It all starts with a Q-table. Think of it as a lookup table that maps every possible state and action to a value.

The process is a simple loop. First, we initialise the Q-table with random values. Then, we iterate through episodes. In each step, the agent observes its current state and uses an epsilon-greedy policy to choose an action. It performs that action, receives a reward, and observes the next state.

The core of the algorithm is the update step. The agent uses a specific rule, called the Bellman equation, to update the Q-value, which helps it learn what the best actions are. This process executes over and over again until the model finds an optimal policy.

But this isn't always easy. Q-Learning faces a few key challenges. First, large state spaces require a huge amount of memory and computation. We often solve this with approximation techniques like Deep Q-Networks. Second, convergence issues can arise, where the model may not reliably find the best solution. Careful tuning of its hyperparameters can help. Finally, as we discussed, balancing exploration and exploitation is a constant challenge, often requiring advanced strategies.

To understand how Q-Learning works, imagine a robot placed in the middle of a five-by-five grid. Each square in the grid is a spot where the robot could be. The goal for the robot is to find its way from its starting position all the way to the target, avoiding any obstacles in the way and using the shortest route it can find.

As the robot moves, it earns different rewards to guide its learning. Reaching the goal gives it a big reward of plus ten. Each time it takes a step, it loses one point, which encourages it to find the shortest possible route. If the robot hits an obstacle, it gets a penalty of minus ten, teaching it to avoid those squares in the future.

At first, the robot doesn't know which actions are best, so it tries different options. As it keeps trying, it updates its Q-values—these are its guesses about the best actions to take from each position. Over many attempts, the robot learns that some moves get it closer to its goal, while others lead to penalties or wasted steps. Through trial and error, it gets better at avoiding obstacles and finding the most efficient path.

Even though this example is quite simple, it captures the key ideas of reinforcement learning: define your states and possible actions, create a reward system that encourages smart choices, and let the agent learn the best strategy by exploring and learning from experience.



Let's look at the steps in Q-Learning and how an agent learns through this process. We start by creating a Q-table—a chart that lists every possible position and action the agent can take. All the values begin at zero.

At each turn, the agent decides which action to try. In the early stages, it might choose randomly but later uses strategies like epsilon-greedy, sometimes exploring and sometimes taking its best-known action.

When the agent makes a move, it observes what happens—did it get closer to its goal, hit an obstacle, or just move to a new spot? It receives either a reward for smart moves or a penalty for mistakes.

The agent then updates its Q-table using what it learned, adjusting its numbers to show which choices work well.

These steps happen again and again across episodes, with the agent gradually improving its strategy, moving from total ignorance to becoming highly skilled in navigating its environment.

Let's talk about how an agent decides on its next move in Q-Learning. The epsilon-greedy strategy guides these choices: most of the time, the agent exploits its knowledge by picking the best-known action, but sometimes it explores by trying something random.

Why do we need this balance? If the agent only explores, it may wander aimlessly and take a long time to improve. If it only sticks to known moves, it risks getting stuck with a strategy that's not truly the best. Mixing both helps the agent avoid dead ends and keeps learning new, smarter ways forward.

For example, imagine our robot starts at position zero-zero. It usually moves right because that's been rewarding before. Occasionally, it tries going up or down instead. These random tries may help uncover a better solution that wasn't possible before.

By carefully balancing curiosity and experience, Q-Learning agents can discover the very best path to reach their goals.

After choosing an action, the agent actually takes that action and watches for the result. Each move changes its position—this is called transitioning to a new state. At the same time, the agent receives feedback in the form of a reward or a penalty, letting it know if its action was good or bad.

For example, imagine our robot starts at position zero-zero in the grid and decides to move right. After making this move, its new state is nought, one. Because moving forward costs energy, the robot receives a small penalty of minus one for the step.

This feedback system is what drives learning. Positive rewards teach the agent to repeat useful actions, while negative rewards encourage it to explore other options. By repeating this process, the agent gradually learns which actions lead to better results in the long term.

Now comes the key moment in Q-Learning—updating the Q-value after each move. This is where the learning truly happens.

We use the Bellman Equation, which lets the agent adjust its knowledge, step by step. In the equation, "s" stands for the current state, "a" is the action just taken, "r" is the reward, and "s-prime" is the new state. Alpha, or the learning rate, tells



us how much weight to give new experience, while gamma, the discount factor, shows how much future rewards matter compared to immediate ones.

Let's see how this works in practice. Suppose the robot starts at position zero-zero and moves right. The immediate reward for that step is minus one. If we use a learning rate of zero point one and a discount factor of zero point nine, then the Q-value update comes out to minus zero point one for that move. This new value will help the agent remember that moving right had a small negative outcome, influencing future choices as it learns.

As the agent repeats this process across many episodes, its Q-values grow more accurate. Gradually, the table fills up with knowledge about which actions lead to rewards and which ones are best avoided—helping the agent find the optimal strategy over time.

Q-Learning doesn't stop with a single attempt. To truly master its environment, the agent needs to repeat the process again and again, running through multiple episodes.

First, the agent explores the environment repeatedly. Every time, it tries out different actions, observes the rewards it receives, and moves through various states. These repeated experiences aren't just busywork—they're essential because the agent builds good knowledge from a wide variety of trials.

With each action and each reward, the agent updates its Q-values. At first, these numbers are just guesses. But as the agent experiences more episodes and receives more feedback, the Q-values become more accurate reflections of which actions lead to better results.

Over time, as the agent keeps updating and refining its Q-table, it's able to learn what's called the optimal policy. This is a clear strategy—it knows, for each state, exactly which action will give it the best long-term reward.

Think of it like practising a sport. At the beginning, you might be clumsy or uncertain. But after many practice sessions where you've tried different moves, learned what works and what doesn't, you start to develop an efficient way to win. Q-Learning agents take that same journey—starting out with random moves, and eventually mastering their environment through repeated learning and improvement.

This process—running lots of episodes, improving Q-values, and extracting the optimal policy—is what makes Q-Learning effective for mastering complex environments.

After many episodes of training, the agent's Q-table starts to show real learning. At first, all the values in the table were zero because the robot had no idea about what to do. But after enough trial and error, you now see higher Q-values for the actions that help the robot reach the goal.

Take a look at this Q-table: for example, in state zero-zero, the Q-value for moving right is now five. While all the other directions stay at zero, this new higher Q-value tells the agent that moving right is clearly its best choice when starting from this



state. In the next state, zero-one, moving right gives a Q-value of four-point-five, again guiding the robot forward along the optimal path.

As the robot continues through the grid, this pattern repeats—high Q-values chart out the most successful sequence of moves. The Q-table now works as a map, showing the robot the most efficient way to reach the goal.

The result is an optimal policy: a strategy learned entirely from experience, where the robot starts out exploring randomly, but ends up following a series of best actions. What began as trial-and-error wandering now becomes a precise, efficient navigation plan, thanks to step-by-step Q-Learning.

If you found Q-Learning fascinating, try building your own agent—start simple and let it learn from mistakes! Remember: The next breakthrough could start with your own experiment!

Video 5- Reinforcement Learning- Demo

Welcome to this session on intuitive concept Exploration versus exploitation. Today we will look into the fascinating sites of exploration and exploitation. And the very heart of reinforcement learning lies one simple but powerful dilemma. Should the agent explore, or should it exploit?

Exploration means trying new actions, ever, even risky or uncertain ones, in order to gather fresh knowledge and possibly discover better strategies. For example, a delivery robot might take a new shortcut it hasn't tried before, even though it's unsure if it will be faster.

Exploitation, on the other hand, is about sticking with what already works, using the best-known action to maximize immediate rewards. That same delivery robot might make it its unusual safe route because it already knows it works reliably. This balancing act is not a one-time decision. Every reinforcement learning agent faces this question at every step it takes in an environment. Too much exploration wastes effort and too much exploitation risk getting stuck with suboptimal choices. This constant trade off truly defines decision making and reinforcement learning.

Let's make this more relatable. Imagine you are deciding where to eat tonight. Exploration is like trying that new Thai place down the street. You don't know what you will get. It could be the best meal of your life, or it could be a total disappointment. But by trying it, you gain new knowledge that might pay off later. Exploitation, on the other hand, is like going to your trusted pizzeria. You know exactly what you will get.

Safe, reliable, and satisfying, but with no surprises and reinforcement learning agent A reinforcement learning agent faces this exact dilemma every single time it takes an action. Should it play safe with what it already knows, or should it venture into the unknown in the hope of finding something better? The choice at its core, reinforcement learning always comes down to the choice. Path A or Path B. Path A is the familiar. This is the safe option. The rewards are known, the outcome is certain, and it feels comfortable. Think of it as going back to your favourite restaurant or taking the same route home every day. You know exactly what you will get. No surprises, but also no new discoveries. Part B is the unknown. This is where exploration happens. The rewards are uncertain, and the outcome could be



surprising. Sometimes it may lead to disappointment, but other times it can open the door to new opportunities and better long-term strategies.

For a real agent is not a onetime decision, it's a decision faced again and again, step after step as it interacts with the environment. Choosing between familiarity and novelty is what shapes how well an agent learns and adopts. Why we need exploration. Exploration is not just about curiosity. It's essential for growth and learning. First, exploration helps us escape local Optima.

Imagine hiking up a hill and thinking you have reached the top, only to later realize there's a much taller mountain nearby. Without exploring further, you would stay stuck with a good enough solution instead of discovering the best one. Second, exploration lets us discover new strategies.

By trying out different actions, an agent can stumble upon novel approaches and hidden opportunities in the environment. Just like a chess player experimenting with new openings, exploration can reveal powerful tactics that were invisible before. And 3rd, exploration is critical to adopt to change environments evolve. What worked yesterday may not work tomorrow.

For example, a stock trading algorithm that only exploits old patterns might fail when the market shifts, but one that explores continuously can adopt and thrive. That's why exploration is not optional. It's a survival skill for both agents and humans. Why do we need exploitation? While exploration is exciting, exploitation is equally important. Without it, an agent could wander endlessly without making real progress first. Exploitation helps to maximize immediate rewards. It makes the most of what the agent already knows to achieve goals efficiently.

For example, if you know 1 vending machine always gives a chocolate bar when you are hungry, you will stick with it rather than risk another machine.

Second, exploitation lets us leverage experience. The agent uses hard earned insights from past trials and successes. Just like a chess player sticking with an opening that consistently wins matches, Exploitation means building and on proven strategies. And 3rd, exploitation helps avoid waste. Constantly trying poor options squanders time and resources. By exploiting what is known to work, an agent ensures efficiency and reliability.

In short, exploitation is what converts learning into consistent results. The delicate balance that we talk about, exploration and exploitation are like two sides of a seesaw. Lean too much on one side and things go wrong. With too much exploration, the agent behave randomly. It wastes time trying things that don't work and never settles on a solid solution. It's like a student constantly switching majors, always learning something new, but never finishing a degree.

With too much exploitation, the agent becomes rigid. It keeps repeating what it knows, even if better options exist. Imagine a business that sticks with the same old product line forever. It avoids risk but eventually stagnates and get gets left behind.

The real success lies in finding the perfect balance, a dynamic approach where the agent explores when it's valuable and exploits when it's when it makes sense. It's like a tightrope Walker leaning too far either way, meaning falling. But with careful balance, progress continues forward.



One of the simplest ways to balance exploration and exploitation is the epsilon greedy strategy. Here is how it works. With a small probability called epsilon, the agent explores by choosing a random action. This ensures it does not get stuck repeating the same things forever. With the remaining probability $1 - \text{Epsilon}$, the agent exploits by choosing the best-known action based on its experience so far. For example, if Epsilon is 0.1, that means the agent will explore about 10% of the time and exploit the best option the other 90%.

It's like being disciplined with your favourite restaurant. You eat there most of the time, but once in a while you deliberately try something new just in case there is a better option out there. The Multi Armed Bandit Game One of the best ways to understand the exploration exploitation dilemma is through the classic multi armed bandit problem.

Imagine you are in a casino facing a row of slot machine machines. Each machine has an unknown payout rate. If you choose pure exploitation, you will keep pulling the same lever every time because it worked once before. The problem is you might miss out on a different machine that actually has a much higher payout, maybe even the jackpot.

On the other hand, pure exploration means you keep trying every machine randomly show you will test everything, but you will also waste a lot of money and time on machine that really pay.

The smarter approach is a balance. Explore different machines at the start to gather information and once you have identified which ones are promising, exploit those. Exploit those more often.

This example is why the multi armed Bandit has become a fundamental model for real-world decision-making problems. From online ads to clinical trials, some real-world applications. The exploration exploitation trade-off is not just theory; it plays out in many real-world system we use every day. Take online recommendations when Netflix or Amazon suggests something new, that's exploration. They are testing if you might like a fresh product, but when they recommend something highly rated and very similar to your past choice says that's exploitation.

Leaning on proven knowledge in self driving car exploration might mean trying out alternative routes during light traffic to see if they are faster. Exploitation, on the other hand, means sticking to the fastest known route during rush hour when reliability matters most.

And in drug discovery, exploration is about testing entirely new molecular combinations, risky but potentially groundbreaking. Exploitation means focusing research dollars on the compounds already showing the most promise.

In each of these cases, success comes from finding the right balance between trying new possibilities and leaning on proven solutions.

Let's summarize the key points we have covered about exploration and exploitation.



First, exploration discovers it, uncovers new possibilities, and prevents agents from settling or settling too quickly for suboptimal solutions.

Second, exploitation optimizes It maximizes return by using existing knowledge and proven strategies, ensuring efficiency in decision making.

Third, remember the balance is dynamic. The right mix of exploration and exploitation changes depending on the context, the stage of learning, and the environment. Successful RL agents adjust this ratio continuously, and finally, strategies enable success. Simple methods like epsilon greedy provide practical ways to manage this trade off, ensuring agents keep learning without losing stability.

Together, these ideas explain why reinforcement learning is not just about curiosity or caution, but about knowing how to combine the two in a smart, adoptive way.

So that was your exploration and exploitation idea.

Thank you.